

HTML LEARNING SERIES

DAY 1

Foundations of the Web & Introduction to HTML

Topics covered today

- What is the Web — building the mental model
 - Web Applications vs Webpages
 - Web Design vs Web Development
- Why HTML — the skeleton of every page
 - HTML document & body structure
 - File extensions & file naming rules
- Basic HTML structure & the most-used tags
- HTML tables — rowspan, colspan, padding & spacing
 - Image tags

Prepared by Ajay M

Web Dev Team Lead
Government Arts and Science College, Palkulam

Contents

1. What Is the Web? (In Your Mind)
 2. Web Applications
 3. Webpages
 4. Web Design
 5. Web Development
 6. HTML Introduction — Why HTML?
 7. How an HTML Document Is Divided
 8. How the Body Section Is Divided (Semantic Layout)
 9. File Extension — Why We Use It
 10. HTML File Extensions
 11. File Naming Rules & Conventions
 12. Basic HTML Structure
 13. Most Commonly Used HTML Tags
 14. HTML Table Tags & Use Cases
 15. Merging Cells — rowspan & colspan
 16. Padding & Spacing in Tables
 17. Image Tags
- Day 1 — Key Takeaways

1. What Is the Web? (In Your Mind)

Before writing a single line of code, it helps to build a clear mental picture of what “the web” actually is. People often use “Internet” and “Web” interchangeably, but they are not the same thing.

Internet: a massive global network of connected computers and servers — the physical roads, cables, satellites, and routers that let data travel from one machine to another.

Web (World Wide Web): a service that runs on top of the Internet. It is a system of linked documents and resources (webpages) that you access using a browser, through the HTTP/HTTPS protocol.

Simple analogy

Think of the Internet as the road network connecting every city. The Web is one of many things you can travel to on those roads — like a city full of shops (websites) you can walk into. Email and online gaming are other “cities” that use the same roads but are not the Web.

When you type a web address and hit Enter, four things quietly work together:

- Browser — the software you use to request and display pages (Chrome, Firefox, Edge).
- URL — the address that identifies exactly which resource you want.
- Server — a remote computer that stores the website's files and sends them back.
- HTML/CSS/JS — the code that tells the browser what to show, how it should look, and how it should behave.

Holding this mental model in place makes everything that follows — webpages, web apps, design, and development — much easier to place in context.

2. Web Applications

A web application (web app) is a software program that runs inside a browser but behaves like a full application — it takes input, processes it, talks to a server/database, and gives back a dynamic, personalized result.

Key characteristics

- Interactive — responds to user actions in real time (clicks, typing, uploads).
- Dynamic — content changes based on data, login state, or user input.
- Connected — constantly communicates with a server/database in the background.
- Often requires login — to save and personalize user data.

Everyday examples

- Gmail — sends, receives, and organizes mail dynamically.

- Instagram / Facebook (web version) — feeds update live, users post and interact.
- Online banking portals — secure, real-time, data-driven.
- Google Sheets — a full application running entirely in the browser.

In short: a web application is something you use to do a task, not just something you read.

3. Webpages

A webpage is a single document that can be displayed in a browser, usually written in HTML. It is the smallest visible unit of the Web — one URL generally points to one webpage.

Key characteristics

- Identified by a unique URL (e.g. `https://example.com/about`).
- Built primarily using HTML for content and structure.
- Can be static (fixed content, same for every visitor) or dynamic (content changes per user/request).
- Multiple webpages linked together form a website.

Webpage vs Website

A webpage is one page (like “About Us”). A website is the whole collection of pages under one domain (Home, About, Contact, etc.), all linked together through navigation.

4. Web Design

Web design is concerned with how a website looks and feels. It is the creative, user-experience-focused layer of building for the web.

What a web designer focuses on

- Layout — how elements are arranged on the page.
- Color scheme — palette, contrast, brand identity.
- Typography — font choice, size, readability.
- Imagery & icons — visuals that support the message.
- Spacing & alignment — whitespace, padding, visual hierarchy.
- User experience (UX) — how easy and pleasant the site is to use.

Common tools

- Figma, Adobe XD, Canva — used to design mockups before any code is written.

Web design answers the question: “How should this look and feel to the user?”

5. Web Development

Web development is the process of actually building the website or web app — turning a design into a working, functional product using code.

The three broad areas

- Frontend development — what the user sees and interacts with directly: HTML, CSS, JavaScript, and frameworks like React, Vue, Angular.
- Backend development — the server-side logic, databases, and APIs that power the site: PHP, Node.js, Python, Java, along with databases like MySQL or MongoDB.
- Full-stack development — a developer who works across both frontend and backend.

Design vs Development

Web design decides how it should look. Web development builds it so it actually works. HTML is the very first tool you pick up on the development side — which is exactly where Day 1 begins.

6. HTML Introduction — Why HTML?

HTML stands for HyperText Markup Language. It is the standard language used to create the structure and content of every webpage on the Internet.

- HyperText — text that contains links to other text/pages (hyperlinks).
- Markup Language — uses tags to “mark up” content, describing what each piece of content is (a heading, a paragraph, an image, a link) rather than performing logic like a programming language.

Why start with HTML?

- It is the skeleton — every website, no matter how advanced, is ultimately rendered as HTML in the browser.
- It is universally understood — every browser on every device can read HTML.
- It works hand-in-hand with CSS (styling/appearance) and JavaScript (behavior/interactivity) — the three form the foundation of frontend development.
- It is beginner friendly — no complex logic, just structure, making it the natural first step into web development.
- It is in constant demand — practically every developer role, from frontend to full-stack, assumes solid HTML fundamentals.

Important note

HTML is a markup language, not a programming language — it has no variables, loops, or conditions. Its job is purely to describe structure and content.

7. How an HTML Document Is Divided

Every HTML document follows the same basic skeleton, split into two major parts inside the `<html>` root element: the `<head>` and the `<body>`.

`<head>` section

Contains information ABOUT the page — metadata that is not directly visible on the page itself, but that the browser and search engines use.

- `<title>` — the text shown on the browser tab.
- `<meta charset="UTF-8">` — defines character encoding.
- `<meta name="viewport">` — controls responsive scaling on mobile devices.
- `<link>` — connects external CSS stylesheets.
- `<script>` — can load JavaScript files.

`<body>` section

Contains everything that is actually visible to the user in the browser window — text, images, links, forms, tables, and every other piece of visible content.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Page Title</title>
  </head>
  <body>
    <!-- visible content goes here -->
  </body>
</html>
```

8. How the Body Section Is Divided (Semantic Layout)

Inside `<body>`, modern HTML gives us semantic tags — tags that describe the meaning/role of a section, not just its position. This makes pages more readable, accessible, and SEO-friendly.

Tag	Role
<code><header></code>	Top section of a page or a section — usually holds the logo, site name, and navigation.
<code><nav></code>	Holds the navigation links (menu) that let users move between pages.
<code><main></code>	The primary, unique content of the page (only one per page).
<code><section></code>	A thematic grouping of related content, usually with its own heading.

Tag	Role
<code><article></code>	A self-contained, independent piece of content (e.g. a blog post, news item).
<code><aside></code>	Side content related to the main content — ads, related links, author bio.
<code><footer></code>	Bottom section — copyright info, contact details, social links.

A typical semantic skeleton looks like this:

```
<body>
  <header>...logo, site title...</header>
  <nav>...menu links...</nav>
  <main>
    <section>
      <article>...page content...</article>
    </section>
    <aside>...related content...</aside>
  </main>
  <footer>...copyright, contact...</footer>
</body>
```

9. File Extension — Why We Use It

A file extension is the short suffix after the dot in a file name (e.g. .html, .jpg, .pdf). It tells the operating system and the browser what type of file it is and which program should open or interpret it.

- Without the correct extension, the computer would not know whether a file is text, an image, a program, or a webpage.
- The browser specifically looks for .html/.htm files to know that it should render the content as a webpage rather than display it as plain text or download it.
- It ensures the file is opened with the right rules — an .html file is parsed for tags, while a .txt file is shown as plain text.

In short

The extension is a promise to the system: “This is what kind of file I am — open me accordingly.”

10. HTML File Extensions

HTML files are commonly saved with one of two extensions:

Extension	Notes
.html	The standard, preferred extension today. Used across all modern HTML5 projects.

Extension	Notes
.htm	An older, 3-character alternative from the days when some operating systems (old Windows/DOS) only allowed 3-letter extensions. Functionally identical to .html in every modern browser.

There is no functional difference between the two in modern browsers — but for consistency, always pick one (.html is the industry standard) and use it throughout a project.

11. File Naming Rules & Conventions

Following consistent, safe naming conventions avoids broken links, server errors, and case-sensitivity bugs, especially once a site is deployed on a Linux server.

- Use lowercase letters only — index.html, not Index.HTML (Linux servers are case-sensitive).
- No spaces — use hyphens instead: about-us.html, not about us.html.
- Prefer hyphens (-) over underscores (_) — search engines treat hyphens as word separators, which is better for SEO.
- Avoid special characters — no @, #, \$, %, &, *, /, or spaces.
- Start with a letter, not a number.
- Keep names short but descriptive — contact.html is better than page2.html.
- Name your homepage index.html — servers automatically look for this file by default.
- Be consistent across the whole project (naming style, casing, folder structure).

12. Basic HTML Structure

Every HTML page, no matter how complex it eventually becomes, starts from this same minimal skeleton:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Page</title>
</head>
<body>

  <h1>Hello, World!</h1>
  <p>This is my first paragraph.</p>

</body>
</html>
```

Line-by-line

- `<!DOCTYPE html>` — tells the browser this is an HTML5 document.
- `<html lang="en">` — the root element; lang helps accessibility tools and search engines.
- `<head>` — metadata, not visible on the page.
- `<meta charset="UTF-8">` — supports almost all characters and symbols.
- `<title>` — text shown on the browser tab.
- `<body>` — everything the user actually sees.

13. Most Commonly Used HTML Tags

Category	Tags	Purpose
Structure	<code><html></code> <code><head></code> <code><body></code>	Root document structure.
Headings	<code><h1></code> ... <code><h6></code>	Section headings, h1 = most important.
Text	<code><p></code> <code>
</code> <code><hr></code> <code></code>	Paragraphs, line breaks, horizontal rule, inline text.
Formatting	<code></code> <code></code> <code><i></code> <code></code> <code><u></code>	Bold, strong emphasis, italics, emphasis, underline.
Links	<code></code>	Hyperlinks to other pages or resources.
Lists	<code></code> <code></code> <code></code>	Unordered list, ordered list, list item.
Media	<code></code> <code><video></code> <code><audio></code>	Embed images, video, and audio.
Containers	<code><div></code> <code><section></code> <code><article></code>	Group and organize content blocks.
Tables	<code><table></code> <code><tr></code> <code><td></code> <code><th></code>	Display tabular / grid data.
Forms	<code><form></code> <code><input></code> <code><button></code> <code><label></code>	Collect user input.

14. HTML Table Tags & Use Cases

Tables are used to display tabular data — information that naturally fits into rows and columns, such as timetables, price lists, mark sheets, or comparison charts.

Tag	Use case
<code><table></code>	Wraps the entire table.
<code><tr></code>	Table Row — defines one horizontal row.
<code><th></code>	Table Header cell — bold & centered by default, used for column/row titles.
<code><td></code>	Table Data cell — holds a normal piece of data.

Tag	Use case
<code><thead></code>	Groups the header row(s) of the table.
<code><tbody></code>	Groups the main body rows of data.
<code><tfoot></code>	Groups footer rows (e.g. totals).
<code><caption></code>	A short title/description shown above the table.

A simple mark-sheet style table:

```
<table border="1">
  <caption>Semester Results</caption>
  <thead>
    <tr>
      <th>Subject</th>
      <th>Marks</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>HTML</td>
      <td>95</td>
    </tr>
    <tr>
      <td>CSS</td>
      <td>90</td>
    </tr>
  </tbody>
</table>
```

15. Merging Cells — rowspan & colspan

Sometimes a cell needs to visually stretch across more than one row or column — for example, a heading that covers two sub-columns. This is done with the `rowspan` and `colspan` attributes.

- `colspan="n"` — makes a cell span `n` columns horizontally.
- `rowspan="n"` — makes a cell span `n` rows vertically.

Example — merging a header across two columns (`colspan`) and merging a name down two rows (`rowspan`):

```
<table border="1">
  <tr>
    <th>Name</th>
    <th colspan="2">Marks</th>
  </tr>
```

```

<tr>
  <th>Term 1</th>
  <th>Term 2</th>
</tr>
<tr>
  <td rowspan="2">Ajay</td>
  <td>85</td>
  <td>90</td>
</tr>
</table>

```

How to remember it

colspan → think “columns” — the cell stretches sideways. rowspan → think “rows” — the cell stretches downward.

16. Padding & Spacing in Tables

Older HTML used two attributes directly on the <table> tag to control spacing. They still work but are considered outdated — modern projects control spacing with CSS instead.

Old HTML attributes (legacy)

- cellpadding — space between a cell's border and its content.
- cellspacing — space between adjacent cells.

```
<table border="1" cellpadding="10" cellspacing="5">
```

Modern CSS approach (recommended)

- padding — applied to <td>/<th> to control internal spacing.
- border-spacing — applied to <table> to control space between cells.
- border-collapse: collapse — merges double borders into a single clean line.

```

<style>
  table { border-collapse: collapse; border-spacing: 0; }
  th, td {
    padding: 10px;
    border: 1px solid #999;
  }
</style>

```

Best practice

Use CSS padding/border-spacing in real projects. The HTML attributes still exist mainly for backward compatibility with very old pages.

17. Image Tags

Images are added to a page using the `<img>` tag. Unlike most HTML tags, `<img>` is a self-closing (void) element — it has no separate closing tag and no visible inner content.

```

```

Common attributes

Attribute	Purpose
src	Path/URL to the image file (required).
alt	Alternate text — shown if the image fails to load, and read aloud by screen readers. Also important for SEO.
width / height	Sets the display size (in pixels) of the image.
title	Extra tooltip text shown on hover.
loading="lazy"	Delays loading the image until it is about to scroll into view — improves page speed.

Best practices

- Always include a meaningful alt attribute — essential for accessibility.
- Don't stretch images unnaturally; keep width/height proportional or control sizing with CSS.
- Prefer relative paths (`images/logo.png`) over absolute local paths so the project stays portable.

Day 1 — Key Takeaways

- The Web is a service built on top of the Internet, made of linked webpages.
- Web apps are interactive/dynamic; webpages are the basic visible documents.
- Design focuses on look & feel; development focuses on building functionality.
- HTML is the structural skeleton of every webpage — not a programming language.
- Every document splits into `<head>` (metadata) and `<body>` (visible content).
- Semantic tags (`header`, `nav`, `main`, `section`, `article`, `aside`, `footer`) organize the body meaningfully.
- File extensions (`.html`) tell the system how to interpret a file; naming should be lowercase, hyphenated, and consistent.
- Tables use `<table>/<tr>/<th>/<td>`, with `rowspan/colspan` to merge cells and CSS `padding/border-spacing` for clean spacing.
- Images are added with the self-closing `<img>` tag — always include alt text.

Coming up on Day 2

Forms & input elements, links & navigation, lists, and an introduction to CSS for styling everything built today.